Лабораторная работа 8 - Создание пользовательского интерфейса на Python

Подготовил студент группы 523 Чех Степан

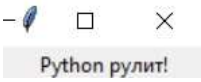
Задание 1. Создать обычное окно в Tkinter

Напишите полный скрипт для создания окна в Tkinter с текстом «Python рулит!». Окно должно выглядеть следующим образом:

Помните, что у каждого программиста есть свой стиль, и ваша программа вовсе не обязана выглядеть точно так же, как код из следующего примера.

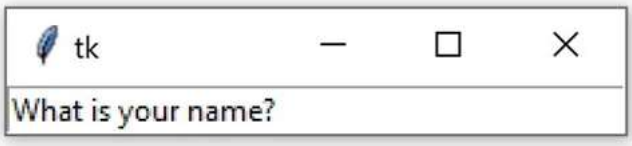
```
import tkinter as tk
# Создаём главное окно приложения
window = tk.Tk()
# Устанавливаем текст заголовка, в верхней рамке окна
window.title("Задание 1. Создать обычное окно в Tkinter")
# Создаём виджет-метку
label = tk.Label(text="Python рулит!")
# Размещаем метку внутри окна с помощью менеджера геометрии
label.pack()
# Запускаем главный цикл обработки событий Tkinter
window.mainloop()
```

Результат



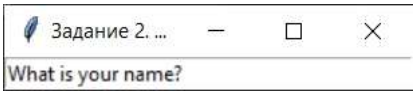
Задание 2. Создать текстовое поле и вставить в нее текст

Попробуйте выполнить данную задачку, чтобы убедиться, что вы действительно поняли принцип работы описанных ранее виджетов.
Задание: Создать однострочное текстовое поле и вставить на нее какой-то текст.
Напишите скрипт для отображения виджета однострочного текстового поля Entry шириной в 40 текстовых юнитов с белым фоном и черным текстом;
Используйте метод .insert() для вставки текста: "What is your name?".
Результат должен выглядеть следующим образом:




```
window = tk.Tk()
window.title("Задание 2. Создать текстовое поле и вставить в нее текст")
# Создаём однострочное текстовое поле ввода
entry = tk.Entry(width = 40)
entry.insert(0, "What is your name?")
# Размещаем поле ввода внутри окна с помощью менеджера геометрии
entry.pack()
window.mainloop()
```

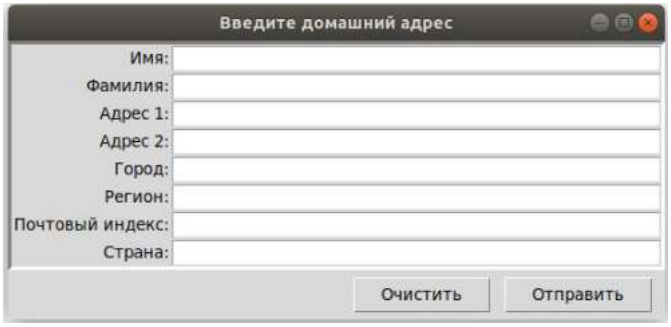
Результат



Задание 3. Создайте анкету с контактными данными клиента

Задание: Создать форму для ввода полного адреса клиента.

Ниже представлено изображение формы для ввода полного домашнего адреса клиента, которая была создана через Tkinter.



Вам нужно написать скрипт для создания такого же окна, что на картинке выше. Используйте любые менеджеры геометрии. После нажатия кнопки отправить, данные должны сохраниться в файл data.txt, а поля формы очиститься.

```
# Функция очистки всех полей ввода
def clear():
    ent_name.delete(0, tk.END)
    ent_surname.delete(0, tk.END)
    ent_address_1.delete(0, tk.END)
    ent_address_2.delete(0, tk.END)
    ent_city.delete(0, tk.END)
    ent_region.delete(0, tk.END)
    ent_post.delete(0, tk.END)
    ent_country.delete(0, tk.END)

# Функция сохранения данных в текстовый файл
def save():
   # Открываем файл в режиме перезаписи ("w"). Если файла нет, он создастся автоматически.
    f = open("client_data.txt", "w")
    # Считываем данные из полей методом .get(), добавляем перенос строки и записываем списком
    f.writelines([ent_name.get() + "\n",
                  ent_surname.get() + "\n",
                  ent_address_1.get() + "\n",
                  ent_address_2.get() + "\n",
                  ent_city.get() + "\n",
                  ent_region.get() + "\n",
                  ent_post.get() + "\n",
                  ent_country.get() + "\n"])
    f.close()

    # Дублируем код очистки полей после сохранения
    ent_name.delete(0, tk.END)
    ent_surname.delete(0, tk.END)
    ent_address_1.delete(0, tk.END)
    ent_address_2.delete(0, tk.END)
    ent_city.delete(0, tk.END)
    ent_region.delete(0, tk.END)
    ent_post.delete(0, tk.END)
    ent_country.delete(0, tk.END)

window = tk.Tk()
window.title("Задание 3. Создайте анкету с контактными данными клиента")
# Создаём два фрейма-контейнера для логического разделения интерфейса
frm_a = tk.Frame()# Будет содержать метки и поля ввода
frm_b = tk.Frame()# Будет содержать кнопки управления

# --- Создание меток (Labels) ---
# Параметр master=frm_a привязывает виджеты к первому фрейму
lbl_name = tk.Label(master = frm_a, text="Имя")
lbl_surname = tk.Label(master = frm_a,text="Фамилия")
lbl_address_1 = tk.Label(master = frm_a,text="Адрес 1")
lbl_address_2 = tk.Label(master = frm_a,text="Адрес 2")
lbl_city = tk.Label(master = frm_a,text="Город")
lbl_region = tk.Label(master = frm_a,text="Регион")
lbl_post = tk.Label(master = frm_a,text="Почтовый индекс")
lbl_country = tk.Label(master = frm_a,text="Страна")
```

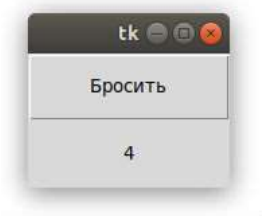
```
# --- Создание текстовых полей (Entry) ---
# width=50 задаёт примерную ширину поля в символах моноширинного шрифта
ent_name = tk.Entry(master = frm_a, width = 50)
ent_surname = tk.Entry(master = frm_a, width = 50)
ent_address_1 = tk.Entry(master = frm_a, width = 50)
ent_address_2 = tk.Entry(master = frm_a, width = 50)
ent_city = tk.Entry(master = frm_a, width = 50)
ent_region = tk.Entry(master = frm_a, width = 50)
ent_post = tk.Entry(master = frm_a, width = 50)
ent_country = tk.Entry(master = frm_a, width = 50)

# --- Размещение меток с помощью менеджера grid ---
# grid использует табличную сетку: row (строка), column (колонка)
lbl_name.grid(row = 0, column = 0)
lbl_surname.grid(row = 1, column = 0)
lbl_address_1.grid(row = 2, column = 0)
lbl_address_2.grid(row = 3, column = 0)
lbl_city.grid(row = 4, column = 0)
lbl_region.grid(row = 5, column = 0)
lbl_post.grid(row = 6, column = 0)
lbl_country.grid(row = 7, column = 0)

# --- Размещение полей ввода в той же сетке ---
# Поля сдвинуты в колонку 1, чтобы выстроиться напротив меток
ent_name.grid(row = 0, column = 1)
ent_surname.grid(row = 1, column = 1)
ent_address_1.grid(row = 2, column = 1)
ent_address_2.grid(row = 3, column = 1)
ent_city.grid(row = 4, column = 1)
ent_region.grid(row = 5, column = 1)
ent_post.grid(row = 6, column = 1)
ent_country.grid(row = 7, column = 1)

# Встраиваем заполненный фрейм анкеты в главное окно (по умолчанию сверху)
frm_a.pack()

# --- Создание кнопок ---
# command=clear привязывает вызов функции при клике
btn_remove = tk.Button(master = frm_b, text = "Очистить", command = clear)
btn_save = tk.Button(master = frm_b, text = "Сохранить", command = save)

# pack(side=tk.LEFT) выстраивает кнопки горизонтально слева направо
btn_remove.pack(side = tk.LEFT)
btn_save.pack(side = tk.LEFT)

# Размещаем фрейм с кнопками под анкетой
frm_b.pack()
window.mainloop()
```

Результат

Задание 4. Создать симулятор броска игральных костей.

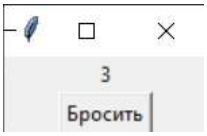
Напишите программу, которая является симулятором результата броска шестигранной игральной кости.
В приложении должна быть кнопка с текстом «Бросить».
При нажатии на эту кнопку должно генерироваться и выводиться на экран случайное число от 1 до 6.
Подсказка: Случайное число можно сгенерировать с помощью randint() из модуля random.

В конечном итоге, приложение должно выглядеть следующим образом:




```
from random import randint
def generate():
    lbl_gambling["text"] = str(randint(1,6))
window = tk.Tk()
window.title("Задание 4. Создать симулятор броска игральных костей")
lbl_gambling = tk.Label(text = "7")
btn_generate = tk.Button(text = "Бросить", command = generate)
lbl_gambling.pack()
btn_generate.pack()
window.mainloop()
```

Результат



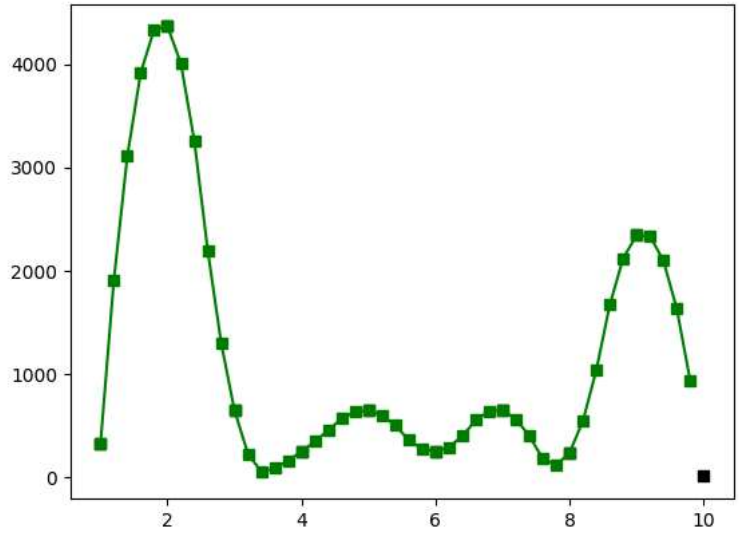
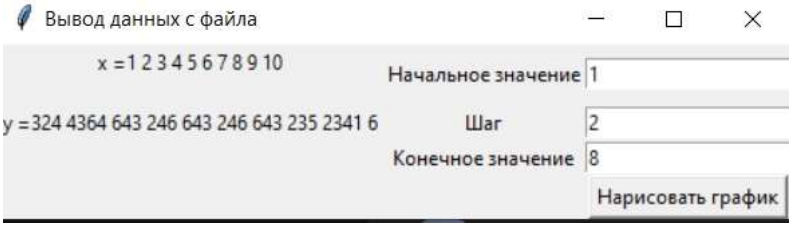
Задание 5. Реализуйте интерфейс для одного из заданий

Окно должно содержать кнопку для открытия выбранного файла.
Содержимое файла после открытия отображается в двух Label или Text (массивы x, y).
Три окна Text позволяют задать начальное значение, шаг, конечное значение для расчёта промежуточных значений (необходима проверка корректности введённых данных).
Результаты расчёта выводятся на графике в новом окне (plt.show): исходные точки отображаются чёрными квадратами, а новые – зелёными квадратами, соединёнными прямыми линиями.

```
from tkinter import filedialog
import matplotlib.pyplot as plt
from functools import partial
import numpy as np
from scipy.interpolate import interp1d

def openfile():
    path = tk.filedialog.askopenfilename(filetypes=[("Текстовые файлы", "*.txt")])
    if path:
        print("Файл выбран:", path)
        interpol(path)

def interpol(path):
    with open(path, "r") as file:
        str_x = file.readline()
        str_y = file.readline()
        x = list(map(int, str_x.split()))
        y = list(map(int, str_y.split()))
    data_window = tk.Tk()
    data_window.title("Вывод данных с файла")

    tk.Label(master = data_window, text = "x =" + str_x).grid(row = 0, column = 0)
    tk.Label(master = data_window, text = "y =" + str_y).grid(row = 1, column = 0)

    tk.Label(master = data_window, text = "Начальное значение").grid (row = 0, column = 1)
    tk.Label(master = data_window, text = "Шаг").grid (row = 1, column = 1)
    tk.Label(master = data_window, text = "Конечное значение").grid (row = 2, column = 1)

    vcmd = data_window.register(is_valid)

    ent_begin = tk.Entry(master = data_window, validate="key", validatecommand=(vcmd, "%P"))
    ent_diff = tk.Entry(master = data_window, validate="key", validatecommand=(vcmd, "%P"))
    ent_end = tk.Entry(master = data_window, validate="key", validatecommand=(vcmd, "%P"))

    btn_draw = tk.Button(master = data_window, text = "Нарисовать график", command = partial(draw_plot, ent_begin,  ent_diff
```

```
    ent_begin.grid (row = 0, column = 2)
    ent_diff.grid (row = 1, column = 2)
    ent_end.grid (row = 2, column = 2)
    btn_draw .grid (row = 3, column = 2)
    data_window.mainloop()

def is_valid(newval):
    if newval == "":
        return True
    try:
        float(newval)
        return True
    except ValueError:
        return False

def draw_plot(e_begin, e_diff, e_end, x, y):
    begin, diff, end = e_begin.get(),  e_diff.get(), e_end.get()
    if begin!= "" and diff!= "" and end!= "":
        print("Числа введены корректно")
        new_x = np.arange(float(begin), float(end), float(diff))
        plt.plot(x, y, 'ks') #Базовый график
        plt.plot(new_x, interp1d(x, y, kind='quadratic')(new_x), 'gs-') #Интерполированный квадратичным
        plt.show()
    else:
        print("Ошибка: введено не число!", begin, diff, end)
        return

file_window = tk.Tk()
file_window.title("Открытие файла")
btn_openfile = tk.Button(master = file_window, text = "Открыть файл", command = openfile)
btn_openfile.pack()
file_window.mainloop()
```

```
Файл выбран: C:/Users/nikso/Desktop/шарага интертеймент/ППП/ppp/какой-то файл я там не понял че делать но вот такое название
Числа введены корректно
```



Результат